

Hadoop Distributed File System (HDFS)

– Basic File Management Exercises

Introduction

This exercise introduces the basic commands used in the Hadoop Distributed File System(HDFS). The aim is to learn how to create directories, upload file, display file contents, check the replication factor, view file size and change the replication factor. These tasks provide practical experience with managing files in HDFS and help build an understanding of how Hadoop stores and replicates data across a cluster for reliability and fault tolerance.

1. **Create /tmp/<yourname> in HDFS**
2. **Upload employees.csv to it**
3. **Print it with -cat — verify all 10 rows**
4. **Confirm replication = 3 in the -ls output**
5. **Check size with -du -s -h**
6. **Change replication to 2 with -setrep and verify**

- The **hdfs dfs -mkdir -p /tmp/Ana** command was used to create a new directory called Ana in the Hadoop Distributed File System (HDFS). The **hdfs dfs -ls /tmp** command was then used to list the contents of the **/tmp** directory and verify that the new folder had been created successfully. The output also displays the directory permissions, owner, group, size, creation date and full path. This confirms that the directory was created correctly and is ready to store files.

```
[consultant@ip-172-31-3-85 ~]$ hdfs dfs -mkdir -p /tmp/Ana
[consultant@ip-172-31-3-85 ~]$ hdfs dfs -ls /tmp
Found 37 items
drwxr-xr-x - hdfs      supergroup      0 2026-07-27 09:52 /tmp/.cloudera_health_monitoring_canary_files
drwxr-xr-x - consultant supergroup      0 2026-07-27 09:52 /tmp/Ana
drwxr-xr-x - hive      supergroup      0 2026-05-11 11:26 /tmp/Yamini
drwxr-xr-x - consultant supergroup      0 2026-07-27 09:48 /tmp/aimunanis
drwxr-xr-x - ec2-user  supergroup      0 2026-05-07 12:15 /tmp/aparn
drwxrwxrwx - ec2-user  supergroup      0 2026-06-16 18:20 /tmp/aparna
drwxr-xr-x - ec2-user  supergroup      0 2026-05-12 19:16 /tmp/cc_fraud_trans
drwxr-xr-x - ec2-user  supergroup      0 2026-05-12 11:56 /tmp/deeksha
drwxr-xr-x - ec2-user  supergroup      0 2026-06-03 08:26 /tmp/ec2-user
-rw-r--r-- 2 ec2-user  supergroup    284 2026-05-07 15:49 /tmp/employees.csv
drwxrwxrwx - ec2-user  supergroup      0 2026-06-22 11:36 /tmp/gokul
drwxrwxrwx - ec2-user  supergroup      0 2026-06-16 13:40 /tmp/gokul_batch
drwxrwxrwx - ec2-user  supergroup      0 2026-06-04 16:38 /tmp/hiren
drwx-wx-wx - hive      supergroup      0 2026-05-15 11:11 /tmp/hive
drwxrwxrwx - hive      hive          0 2026-05-18 09:20 /tmp/itc_runbook
drwxr-xr-x - consultant supergroup      0 2026-07-27 09:50 /tmp/itc_sreenirunbook
drwxr-xr-x - consultant supergroup      0 2026-07-27 09:17 /tmp/joshua
drwxrwxrwt - mapred    hadoop        0 2026-06-17 11:27 /tmp/logs
drwxr-xr-x - consultant supergroup      0 2026-07-13 10:33 /tmp/om
drwxr-xr-x - ec2-user  supergroup      0 2026-06-02 13:36 /tmp/project
drwxrwxrwx - ec2-user  supergroup      0 2026-06-10 17:35 /tmp/samia
drwxr-xr-x - ec2-user  supergroup      0 2026-05-08 10:55 /tmp/samia_employees
drwxr-xr-x - ec2-user  supergroup      0 2026-05-06 13:19 /tmp/samiaa
drwxr-xr-x - ec2-user  supergroup      0 2026-05-07 16:00 /tmp/samiaaaa
drwxr-xr-x - ec2-user  supergroup      0 2026-05-07 15:47 /tmp/samiaaaa
drwxr-xr-x - ec2-user  supergroup      0 2026-05-12 16:31 /tmp/samiya
drwxr-xr-x - consultant supergroup      0 2026-07-26 18:49 /tmp/sruthi
drwxrwxrwx - ec2-user  supergroup      0 2026-06-12 09:35 /tmp/subirna
```

- The **cat** command was used with a here document (<<EOF) to create a CSV file named **employees.csv** in the **/tmp** directory. The first line defines the column headers (ID, name, age, city and salary), while the remaining ten lines contain employees records. Typing EOF ends the input and saves the data to the file. This method provides a quick way to create a structured CSV file directly from the Linux terminal without using the text editor.

```
[consultant@ip-172-31-3-85 ~]$ cat > /tmp/employees.csv << 'EOF'
> id,name,age,city,salary
> 1,Alice,28,London,45000
> 2,Bob,34,Manchester,52000
> 3,Charlie,25,Birmingham,39000
> 4,David,40,Leeds,61000
> 5,Emma,30,Bristol,48000
> 6,Frank,29,Liverpool,47000
> 7,Grace,35,Sheffield,55000
> 8,Harry,32,Nottingham,51000
> 9,Ivy,27,Leicester,42000
> 10,Jack,38,Coventry,58000
> EOF
```

- The **hdfs dfs -put** command was used to upload the employees.csv file from the local Linux /tmp directory to the **/tmp/Ana/** directory in HDFS. After the upload, the **hdfs dfs -ls** command was used to verify that the file had been stored successfully. The output confirms the file's permissions, replication factor of 3, owner, group, file size, upload, date and time and its location in HDFS. This verifies that the upload was successful and the file is ready for use within the Hadoop environment.

```
[consultant@ip-172-31-3-85 ~]$ hdfs dfs -put /tmp/employees.csv /tmp/Ana/
[consultant@ip-172-31-3-85 ~]$ hdfs dfs -ls /tmp/Ana/
Found 1 items
-rw-r--r-- 3 consultant supergroup      284 2026-07-27 09:56 /tmp/Ana/employees.csv
[consultant@ip-172-31-3-85 ~]$
```

- The **hdfs dfs -cat** command was used to display the contents of the **employees.csv** file stored in HDFS. The output shows the column headers followed by ten employees records, including each employee's ID, name, age, city and salary. Displaying the file confirms that it was uploaded successfully and that all data is present and readable. This verification ensures the integrity of the file before it is used for further Hadoop processing.

```
[consultant@ip-172-31-3-85 ~]$ hdfs dfs -cat /tmp/Ana/employees.csv
id,name,age,city,salary
1,Alice,28,London,45000
2,Bob,34,Manchester,52000
3,Charlie,25,Birmingham,39000
4,David,40,Leeds,61000
5,Emma,30,Bristol,48000
6,Frank,29,Liverpool,47000
7,Grace,35,Sheffield,55000
8,Harry,32,Nottingham,51000
9,Ivy,27,Leicester,42000
10,Jack,38,Coventry,58000
[consultant@ip-172-31-3-85 ~]$
```

- The **hdfs dfs -du -s -h /tmp/Ana/** command was used to check the size of the directory stored in HDFS. The **-du** option displays disk usage, **-s** provides a summary for the directory and **-h** formats the output in a readable way. The results show that the actual file size is 284 bytes, while 852 bytes of HDFS storage are used because the file was a replication factor of 3. This demonstrates how HDFS replicates data to improve reliability and fault tolerance.

```
[consultant@ip-172-31-3-85 ~]$ hdfs dfs -du -s -h /tmp/Ana/
284 852 /tmp/Ana
[consultant@ip-172-31-3-85 ~]$ hdfs dfs -get /tmp/Ana/employees.csv /tmp/employees_copy.csv
```

- The **hdfs dfs -cp** command was used to create a backup copy of the **employees.csv** file within the HDFS directory. The **hdfs dfs -ls** command was used to verify that both the original file and the backup file were present. Finally, the **hdfs dfs -setrep 2** command changed the replication factor of **employees.csv** from 3 to 2. This demonstrates how HDFS allows files to be copied and how the number of replicas can be adjusted to balance data reliability and storage usage.

```
[consultant@ip-172-31-3-85 ~]$ hdfs dfs -cp /tmp/Ana/employees.csv /tmp/Ana/employees_backup.csv
[consultant@ip-172-31-3-85 ~]$ hdfs dfs -ls /tmp/Ana/
Found 2 items
-rw-r--r-- 3 consultant supergroup 284 2026-07-27 09:56 /tmp/Ana/employees.csv
-rw-r--r-- 3 consultant supergroup 284 2026-07-27 10:08 /tmp/Ana/employees_backup.csv
[consultant@ip-172-31-3-85 ~]$ hdfs -setrep 2 /tmp/Ana/employees.csv
```

hdfs dfs -mkdir -p /tmp/Ana/ → Creates a directory named **Ana** in HDFS. The **-p** option creates parent directories if needed and avoids errors if the directory already exists.

hdfs dfs -ls /tmp → Lists all files and directories inside the **/tmp** directory in HDFS. Used to verify that the **Ana** directory was created.

Cat > /tmp/employees.csv << EDF → Creates a new CSV file called **employees.csv** in the local Linux **/tmp** directory and allows multiple lines of text to be entered until EOF is typed.

Hdfs dfs -put /tmp/employees.csv/tmp/Ana/ → Uploads the local **employees.csv** file into the **/tmp/Ana** directory in HDFS.

Hdfs dfs -ls /tmp/Ana/ → Lists all file in the **/tmp/Ana/** directory. Used to verify that the file was uploaded successfully and to display its replication factor, owner, size and date.

Hdfs dfa -cat /tmp/Ana/employees.csv → Displays the contents of the **employees.csv** file directly from HDFS. Used to verify that all 10 records were uploaded correctly.

Hdfs dfs -du -s -h /tmp/Ana/ → Shows the size of the directory. **-du** displays disk usage, **-s** provides a summary and **-h** makes the output human-readable.

Hdfs dfs -get /tmp/Ana/employees.csv /tmp/employees_copy.csv → Downloads the file from HDFS back to the local Linux system and saves it as **employees_copy.csv**.

Hdfs dfs -setrep 2 /tmp/Ana/employees.csv → Changes the replication factor of the file from 3 to 2, reducing the number of copies stored in HDFS.

Hdfs dfs -ls /tms/Ana/ → Runs after **-setrep** to verify that the replication factor had changed from 3 to 2.

Conclusions

This practical exercise provided a basic introduction to working with Hadoop Distributed File System (HDFS). Throughout the tasks, I learned how to create directories, upload files, verify file contents, check file sizes, create backup copies and change the replication factor of files. I also learned how to use common HDFS commands such as **-mkdir, -put, -ls, -cat, -du, -cp, -get and -setrep.**

By completing these activities, I gained a better understanding of how HDFS stores and manages data across a Hadoop cluster. I also learned the importance of data replication for fault tolerance and how changing the replication factor affects storage usage. Overall, this exercise helped me develop practical skills in managing files within the HDFS and provided a strong foundation for more advanced Hadoop and big data tasks.